Case Study: Agent World Model

1,000 synthetic, code-driven environments for agentic RL

Wang, Xu, Liu, Wang, Han, Yao, Yao, He · UNC Chapel Hill and Snowflake · ICML 2026

Learning outcomes

By the end of this session, you can:

1. Explain how AWM generates an executable environment from only a website name.
2. Map AWM's database, tools and checks onto the parts of a POMDP.
3. Explain why AWM combines verification code with an LLM judge.
4. Describe AWM's GRPO setup: reward, format penalty, history window.
5. Read AWM's results and limits with care.
6. Plan an AWM-style environment for an Indian task.

The problem: too few practice worlds

Real apps

Many: no public API

RL: thousands of calls per task

Slow, costly, not resettable

Hand-built benchmarks

τ^2-bench: 3 environments

TheMCPCompany: 5

Good tests, too small for RL

LLM-simulated apps

An LLM invents each tool response

Hallucinated, inconsistent state

One LLM call per step

Missing: environments that are many, diverse, executable and reliable.

Build each practice app the way a software team does. Then let agents train inside it.

Requirements → database → API → tests · all written by an LLM, all executed and checked

AWM at a glance

1,000
environments
from 100 site names

10,000
tasks
10 per environment

35,062
tools
~35 per environment

~2,000
lines of Python each
FastAPI + SQLAlchemy

18.5
SQLite tables each
129 sample rows

8.5
agent steps per task
7.1 distinct tools used

Each environment is a POMDP

POMDP part	In AWM	Spotify example
State S	The SQLite database	25 tables: tracks, playlists…
Action A	One MCP tool call	create_playlist(name=…)
Observation O	The tool's response	JSON: new playlist id 41
Transition T	Python code running SQL	INSERT INTO playlists …
Reward R	Verification per task	Track now in playlist?

Database → S · interface → A, O, T · verification → R

“Partially observable”: the agent sees only replies

Hidden from the agent

Full database, all 25 tables

Other users' playlists

Rows no tool returns

Which tools exist, until it asks

Visible to the agent

Task text plus tool replies

search_tracks → 3 matches

get_playlists → ids and names

Errors: “500 Internal Server Error”

Analogy: a bank clerk sees only the screens a search returns, never the whole core-banking database.

MCP: one plug for all 1,000 apps

MCP = Model Context Protocol

Anthropic's standard for agent tools

Only two meta-tools

list_tools: names, schemas, descriptions

call_tool(name, arguments)

Why

One agent, any app, no hard-coding

Benchmarks adapted through call_tool

```
{"name": "list_tools"}

{"name": "call_tool",
 "arguments": {
  "tool_name":
    "get_weather",
  "arguments":
    "{\"city\": …}"}}
```

Part 1

The generation pipeline

From a website name to a working app with tasks

Five steps, each built from the last

1
Scenario
Name + 150–300 word brief

2
Tasks
10 user requests

3
Database
Schema + sample rows

4
Interface
API spec, then MCP code

5
Verification
Check code per task

Order matters: tasks decide tables · tables decide tools · tools decide checks

Every step: run the output · on error, fix and retry

Step 1 · 100 names grow into 1,000 apps

Seed: 100 popular site names

Similarweb list: Amazon, Gmail, GitHub

Expand: Self-Instruct style

LLM writes new apps from a few examples

Filter 1: CRUD classifier

Create, read, update, delete records?

Filter 2: dedup, cosine ≥ 0.85 out

Filter 3: caps on big categories

Generated scenarios

ClaimWise Claims Desk

TeleVisit360 Virtual Care

GrantPath Scholarships

KitchenGrid Restaurant Ops

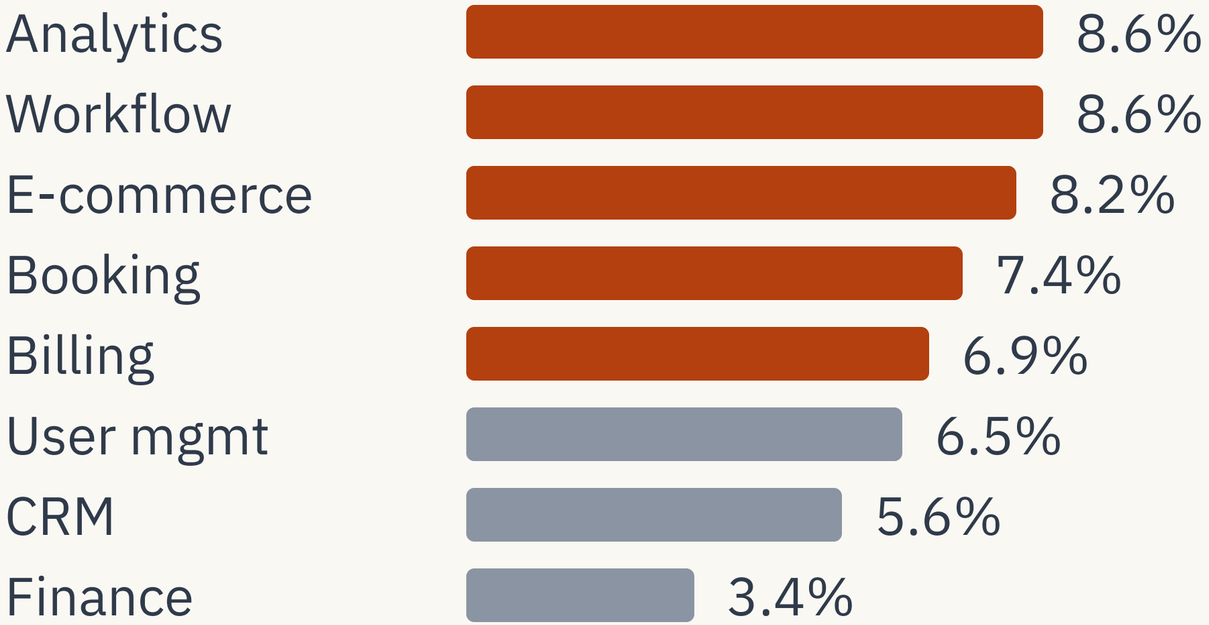
Toast POS · Bill.com · Jira

The test: can the data be faked usefully?

Suitability	Examples	Reason
High	E-commerce, banking, HR/payroll, healthcare appointments, logistics	Rows, statuses, numbers
Medium	YouTube, Spotify, Reddit	Playlists yes, audio no
Low: avoid	News, Wikipedia, search, ChatGPT, Google Translate	Content is the product

Fakeable: prices, orders, statuses, dates, cities · **Not:** articles, media, AI replies, ranking

What the 1,000 apps cover



Share of 6,715 tags

One app, several tags

“Other”: 16.8%

Topics: 3,619 at 1,000 apps

Step 2 · Ten tasks act as the requirements

API-solvable: no clicks, no page visits **Logged in:** user_id = 1

Self-contained: every name, date and number in the task

> **Music:** Create playlist “Morning Focus 2025”, add top 10 Daft Punk tracks
>
> **Shop:** Return my “Instant Pot Duo 7-in-1” from the last 6 months; refund to original payment
>
> **Travel:** Pet-friendly Lisbon flat, 5 Jan–5 Feb 2026, ≤ €80/night, top 3 by rating

Step 3 · The database is the state

Schema (SQLite DDL)

Only tables the tasks need

“Cancel order” → Orders + status

Keys, foreign keys, indexes

No password or token fields

Why SQL: keys enforce consistency

Sample data (INSERTs)

Meets each task's preconditions

“Update stock” → a product with stock

Search tasks: matches and non-matches

List tasks: 5–10 records

Delete tasks: rows safe to delete

Accept rule: under 10% failed statements · **Average:** 18.5 tables, 129 rows

Example: the generated “Spotify” database

```
class Playlist(Base):
  __tablename__ = "playlists"
  id = Column(Integer,
    primary_key=True)
  owner_user_id = Column(
    ForeignKey("users.id"))
  name = Column(Text)
  is_collaborative = …
```

25 tables

users, artists, albums, tracks

playlists, playlist_tracks

listening_history, moods

playback_sessions, settings

~2,400 lines of code

45 MCP tools

Step 4 · Interface: write the spec, then the code

4a · API spec (JSON)

Minimal endpoints for the 10 tasks

Small, atomic, composable

Typed params, response schema

Doubles as docs for the agent

4b · One Python file

FastAPI + SQLAlchemy + Pydantic

Every endpoint = one MCP tool

Filters by user_id = 1

Check: server starts, list_tools works

Why two stages: 30+ tools in one shot → inconsistent code · apps can exceed 3,000 lines

Example: what list_tools returns for Spotify

```
22. mcp_tool_create_playlist
  Create a new playlist owned by the user.
  - name: string (required, unique per user)
  - description: string (optional)
  - is_collaborative: boolean (default False)
```

Also: search_artists, get_artist_top_tracks, get_top_tracks_grouped_by_genre

45 tools in this app · 35 on average

Self-correction: run it, read the error, retry

1 Run the output in a sandbox

2 Error → full traceback

3 2nd LLM call: 200–500 token summary

4 Regenerate with the summary

At most 5 retries per part

Limits: 10% schema/data, 0% server

Stage	Pass	Cost
Database	88.3%	$3.59
Sample data	88.2%	$13.75
Tool spec	–	$23.74
Env code	86.8%	$12.81
All stages	–	**$57.09**

Result: >85% work first time · failed parts need 1.13 retries on average

AWM is 5× larger than the nearest rival

Set	Human input	SQL	Envs	Tools
τ²-bench	Hand-built	No	3	22.7
MCP-Universe	Real APIs	–	11	12.1
AutoForge	Tool docs	No	10	–
EnvScaler	Task sets	No	191	18.6
AWM	**Names only**	**Yes**	**1,000**	**35.1**

Tools = average per environment · code: 1,985 lines vs 662 (EnvScaler)

Part 2

Verification

Is the app correct? Did the agent finish the task?

Two different checks

A · Is the app correct?

When: during generation

How: execute each layer

Schema loads, data inserts

Server starts, list_tools answers

Audit: LLM judges score 100 apps

B · Did the agent succeed?

When: after every rollout

How: check code + LLM judge

Database before vs after

Judge reads the trajectory too

Output: the RL reward

Step 5 · Check code: before vs after

Task: Add “Blinding Lights” by The Weeknd to my playlist “Driving Vibes”

```
def verify_task(initial_db,
                final_db):
  before = tracks_in(initial_db)
  after  = tracks_in(final_db)
  return {"added": song in after
        and song not in before,
   "tracks_after": len(after)}
```

Read-only SQL on both

Returns evidence, not a verdict

Plus success and failure criteria in words

The code-augmented LLM judge decides

Judge: GPT-5

Input 1: task + full trajectory

Input 2: check-code output + criteria

Told: checks may be empty or wrong

Returns: reasoning, label, evidence

Label (priority order)	Reward
1 Completed	1.0
2 Environment Error	0
3 Agent Error	0
4 Partially Completed	0.1

Why not code alone: flaky servers, idempotent tasks · WebArena, τ^2, SWE-bench all fixed misjudged graders

Three cases: why both checks are needed

What happened	Code only	Judge only	Both
Auction: correct bid history returned	Right	Right	Right
Smart home: create gives 500 error; routine already exists	**Wrong: “no change”**	Right	Right
Events: agent makes a duplicate event, edits that one	Right	**Wrong: “success”**	Right

Rule: code gives evidence from state · judge gives context from the trajectory

Code + judge gives the best reward

8B, reward from	BFCL	τ² P@1	MCP
LLM judge only	55.5	26.4	10.6
Code only	60.0	29.6	5.6
Code + judge	**65.9**	**33.5**	**11.2**

Judge reliability

100 trajectories × 5 runs

GPT-5.1: 95.5% agreement

Fleiss' κ 0.891

Reward flips: 9.2%

Open Qwen3.5: 88.1%

Same ranking at 4B and 14B · extra judge cost ≈ $1.80 per step

The apps have bugs, but few block training

74%

apps with ≥ 1 bug

EnvScaler: 88%

14%

tasks blocked by bugs

EnvScaler: 57.1%

~4%

rollouts hit an error during RL training

Bug types: unhandled edge cases 44% · database constraint clashes 14%

Defence: “Environment Error” label → reward 0, not a false failure

Part 3

Agentic RL with GRPO

Reward design, format penalty, history-aware training

Reward: task outcome plus a format penalty

End of rollout: outcome

Completed → **1.0**

Partially Completed → **0.1**

Anything else → **0**

Copied to every step

Each step: format

No <think> block

Invented tool name, bad JSON

list_tools not first, or twice

Any violation → **−1, stop now**

Server error (timeout, 5xx) → reward 0, not −1 · do not punish the agent for the app

The format penalty speeds up learning

Measure (first ~50 steps)	With penalty	Without
Format error ratio	Falls to ~0.1–0.2	Stays above 20%
Task completion	Rises to ~0.5 (8B, 14B)	Stuck below 40%
Rollout time	27% shorter	Baseline

Lesson: in long agent episodes, stop broken rollouts early · cheaper and cleaner signal

Train on the history the agent will see

Problem

Deploy: last few turns only

Usual RL: whole history, one pass

→ train/test mismatch

Fix

Window w = 3 recent turns

Rollout of T turns → T samples

Loss only on turn t's tokens

4B model	BFCL	τ² P@1
Aligned, window	**64.5**	**22.6**
Aligned, full	55.4	15.9
Misaligned, window	61.9	9.4

Each sample: system prompt + task + list_tools reply + last 3 turns

Training setup: 1,024 live apps per step

GRPO setting	Value
Models	Qwen3 4B, 8B, 14B
Train data	526 apps, 3,315 tasks
Batch × rollouts	64 × 16 = 1,024
Steps · learning rate	≤ 96 · 7×10^{-7}
KL · clip-high	0.001 · 0.28
Max turns · window	20 · 3

Infra

AgentFly + verl

Own MCP server + DB copy each

Reset = restore the database

Prefetch next batch's apps

Judge calls: async

Part 4

Results

Trained only on synthetic apps, tested on three unseen benchmarks

Three tests, three baselines

Benchmarks

BFCLv3 function calling, multi-turn, hallucination

τ²-bench airline, retail, telecom; simulated user

MCP-Universe real MCP servers: maps, finance, browser

Baselines

Base Qwen3, no extra training

Simulator same tasks, GPT-5 plays the app

EnvScaler 191 synthetic apps, SFT + RL

Not trained for: chatting with a user, refusals, browsing — all tested here

AWM improves on all three unseen tests

Method	BFCL 8B	τ² P@1 8B	MCP 8B	BFCL 14B
Base Qwen3	53.8	26.4	6.7	61.3
Simulator	52.5	31.3	6.2	67.7
EnvScaler	36.8	**39.4**	5.6	–
AWM	**65.9**	33.5	**11.2**	**70.2**

Only AWM beats Base everywhere · EnvScaler drops on BFCL (–8.9 avg) and MCP

Big gains on simple tasks, small on hard ones

BFCL, 8B	Base	AWM	Gain
Simple	53.6	80.3	**+26.7**
Medium	60.0	75.3	**+15.3**
Hard	43.9	45.0	**+1.1**

τ², 8B

Simple +9.3 · hard +4.5

A cost

Hallucination category, 8B: 76.4 → 70.8

Cause: always rewarded for calling tools

More environments, better agents

4B, trained on	BFCL	τ² P@1	MCP
10 AWM apps	48.0	14.2	3.9
100 AWM apps	57.0	16.6	6.1
191 AWM apps	57.6	16.4	6.1
191 EnvScaler apps	54.1	29.0	4.5
526 AWM apps	**64.5**	**22.6**	**6.7**

10 apps: overfits, below Base (54.9) · **still rising at 526**

The synthetic tasks are not easy

Tool calls	Sonnet 4.5	Neither passes
1–3	68.4%	15.2%
4–6	71.8%	26.2%
7–10	63.3%	36.3%
11+	31.0%	69.0%

“Neither” = neither GPT-5.1 nor Sonnet 4.5 passes

Overall Pass@1

Claude Sonnet 4.5: 62.6%

GPT-5.1: 36.1%

Never solved in RL

24–28% of tasks

Limits to keep in mind

Fixed pipeline

No self-evolving tasks or apps

Imperfect apps

74% with bugs; fixes by trial and error

No adversaries

No injected prompts or bad rows

Partial training

526 apps; Qwen3 family only

Always logged in

No access-control training

Judge = GPT-5

Judges agree; no human study

Part 5

An AWM for India

Indian systems, Indian rules, Indian languages

Eight ideas to copy from AWM

1 Names as seeds → expand, filter, dedup

2 Tasks first → tables follow tasks

3 SQL as state → keys keep it consistent

4 Spec, then code → fewer broken tools

5 Run and repair → error summary, retry

6 DB diff + judge → 4 labels

7 Stop bad steps → −1, end rollout

8 Train as deployed → same history window

All eight work for Indian apps unchanged · what changes: seeds, rules, language

Indian seed scenarios

Scenario	Example task
Rail booking	2 Tatkal seats Pune → Nagpur; 3AC if waitlist > 20
UPI dispute desk	Failed ₹2,450 debit: dispute, then compensation
GST filing desk	October GSTR-3B; credit only for 2B invoices
CSC scheme desk	Farmer 42: PM-KISAN? Apply, or record why not
Hospital claims	Map surgery to a package; send pre-authorisation
Court registry	Adjourn to next working day; update cause list

All pass AWM's test: records, statuses, amounts, dates

Worked example: a GST filing desk

State (tables)

invoices, gstr2b_entries, itc_ledger, returns, challans

Tools (MCP)

get_gstr2b, match_itc, file_gstr3b, pay_challan

Sample data

2 invoices missing from 2B · 1 blocked credit

Check code

ITC = matched sum, to the rupee · challan = net tax

Task: File October 2026 GSTR-3B. Claim credit only for invoices in GSTR-2B. Pay the balance.

Add the language layer

One state, many surfaces

Task in Hindi, Hinglish, Tamil

Same database, same check code

Check the arguments

Main Indic error: wrong values

“Pune” vs “पुणे” vs “Poona” in a field

Add a simulated user

AWM has none; τ²-bench style

Hinglish customer, partial details

Score the reply language

Reply in the user's language

Report the English-vs-Indic gap

> “Bhaiya, kal ka ₹2,450 UPI fail hua par paisa kat gaya, refund kab tak?”

Fix AWM's gaps for Indian work

AWM gap	Indian fix
Always logged in	KYC, consent, OTP-failure tasks
No refusal tasks	Fraud and out-of-policy requests: reward a refusal
Rules fixed in code	Rule tables with dates: GST 2.0, TDS from 1 Apr 2026
Sample data	Fake IDs, valid checksums: Aadhaar, GSTIN, PAN
English judge	Test judge vs native speakers first
Real-app names	Clones only; never touch live portals

A research project: “Bharat AWM”

1 Build 50–100 Indian apps × 10 tasks · AWM repo, Indian seeds

2 Audit 20 apps by hand · bugs, blocked tasks, rule errors

3 Baseline API models + Indian models · pass@1, pass^4, per language

4 Train Qwen3-4B, GRPO + LoRA · smaller groups than AWM

5 Test transfer held-out Indian apps · BabelArena · BFCL · τ²-bench

6 Release apps, tasks, check code, judge prompts, report

Open questions: Does Hinglish training help English? Does judge agree with humans in Tamil?

Questions for discussion

1. How could an agent fool the code-plus-judge reward?
2. Why did training on 10 apps score below the base model?
3. Is 0.1 for “Partially Completed” a good choice?
4. Would you train a UPI agent in an LLM-simulated bank?
5. What does “always logged in” hide in Indian banking?

Where to read in the paper

Topic	Sections, figures, tables
Generation	Fig 2 · §3.1–3.3 · App A.1–A.3 · Fig 10, 12, 15
App correctness	Self-correction (p.5) · Table 1 · §6.1, Tables 6–7
Task verification	§3.3.1 · Figs 21–22, 26–27 · §6.2 · App B.4
POMDP, MCP, RL	§3 opening · App A.4 · §4.1–4.2 · App A.5, Table 12
Dataset	Tables 2, 3, 15, 17 · Figs 6, 8, 25 · §5.1
Results	Tables 4, 5, 8, 9, 11 · Fig 4 · Limitations

Read the originals

- Paper: Agent World Model (ICML 2026)
- Code and 1,000 environments
- EnvScaler · τ²-bench · MCP-Universe
- GRPO (DeepSeekMath) · DAPO
- Model Context Protocol